SQL Data Analysis: Querying the Orders Database

Project 3: SQL Data Analysis | Database: SQLite (orders.db), table **orders** | 1,200 orders, 2023-01-01 to 2025-06-30 | 21 queries

Executive Summary

1. Problem Statement

What can structured SQL queries (filter, group, aggregate) tell us about orders, revenue, order status and customers, and do the SQL answers agree with the earlier Python analysis?

2. Methodology

Input: the cleaned Project 1 dataset, loaded into a SQLite table with a PRIMARY KEY, UNIQUE and CHECK constraints. **Process:** 21 queries in six sections: overview, WHERE filtering, GROUP BY aggregation, time trends, HAVING and subqueries, and business insight. **Output:** a results workbook, this report, and a verification step that re-computes 14 key numbers in pandas and checks that SQL and Python agree (14/14 match).

3. Key Findings

- **41.1% of order value ($519,674) is cancelled or returned; only 19.2% is delivered.** (Q09)
- **First-half revenue fell 10.3% in 2024 and 9.8% in 2025.** (Q14)
- **A typical order is $823.62, not $1,053.97.** 40.9% of orders exceed the average. (Q18, Q19)
- **No product, payment method, channel or coupon stands out.** The product cancel-rate gaps are consistent with chance (p = 0.98); coupon orders average $14.27 more, within noise. (Q07, Q10-Q12, Q16)
- **Only 11 of 1,189 customers ordered twice.** (Q17)
- **All 14 SQL results match the independent pandas calculation**, and the totals reconcile with the Project 2 report.

4. Recommendations

- **Investigate cancellations and returns first.** Add reason codes so the next query can show why, not just how much.
- **Report the median order value** (Q19 is a reusable query) alongside the average.
- **Do not act on product-level cancel-rate gaps** until they pass a significance test; keep the HAVING query as a monitor.
- **Turn the monthly and first-half queries (Q14, Q15) into saved views** so leadership sees like-for-like revenue every month.
- **Verify the status and customer fields at the source** (they look unusually even and unique), then re-run the same queries.

Requirements Coverage

Project requirement	How it is demonstrated	Queries
SELECT queries	Column lists, aliases, DISTINCT, LIMIT, expressions	Q01 to Q21
WHERE	Comparison (>, <>), BETWEEN, AND, LIKE, IN, date filters	Q04, Q05, Q06, Q14
ORDER BY	Ascending and descending, by aliases and aggregates	Q01, Q04, Q07, Q15
GROUP BY	One column, several columns, and computed expressions (year, month, CASE)	Q06 to Q15, Q20, Q21
COUNT, SUM, AVG	Plus MIN, MAX and COUNT(DISTINCT)	Q02, Q07 to Q13, Q21
HAVING (slides: conclusion)	Filtering grouped data	Q03, Q16, Q17
Percentage contribution (slides: conclusion)	Group total divided by overall total	Q07, Q08, Q09
Beyond the brief	CASE, subqueries, a CTE with a window function (LAG), median in pure SQL, date functions	Q09, Q14, Q16, Q18, Q19

The Database

The Excel data was loaded into one table with constraints, so the database itself rejects bad data (a duplicate OrderID or tracking number, a zero price, a missing status).

Column	Type	Constraint	Column	Type	Constraint
OrderID	TEXT	PRIMARY KEY	**PaymentMethod**	TEXT	NOT NULL
Date	TEXT	NOT NULL (ISO YYYY-MM-DD)	**OrderStatus**	TEXT	NOT NULL
CustomerID	TEXT	NOT NULL	**TrackingNumber**	TEXT	UNIQUE
Product	TEXT	NOT NULL	**ItemsInCart**	INTEGER	
Quantity	INTEGER	NOT NULL, CHECK > 0	**CouponCode**	TEXT	NOT NULL ("No Coupon" when none)
UnitPrice	REAL	NOT NULL, CHECK > 0	**ReferralSource**	TEXT	
ShippingAddress	TEXT		**TotalPrice**	REAL	NOT NULL

How the Database Reads a Query

A query is typed starting with SELECT but executed in a different order. Every query in this project was written with that execution order in mind.

Step	Clause	What it does	Example from this project
1	**FROM**	Locates the data source	FROM orders (1,200 rows)
2	**WHERE**	Filters individual rows	WHERE TotalPrice > 3330.41 keeps 8 rows (Q04)
3	**GROUP BY**	Sorts rows into buckets	GROUP BY Product makes 7 buckets (Q07)
4	**HAVING**	Filters whole buckets	HAVING COUNT(*) > 1 keeps 11 customers (Q17)
5	**SELECT**	Picks columns and computes aliases	SUM(TotalPrice) AS order_value is created here
6	**ORDER BY**	Sorts the final result	ORDER BY order_value DESC can use the alias

The alias trap. *WHERE runs before SELECT, so an alias like rev does not exist yet. SQLite is lenient and accepts it (tested: the query ran), but MySQL, PostgreSQL and SQL Server reject it. These queries repeat the full expression in WHERE and HAVING so they stay portable.*

Verification: SQL versus Python

Check	SQL result	pandas result	Match
Total orders	1,200	1,200	**PASS**
Unique customers	1,189	1,189	**PASS**
Total order value	1,264,761.96	1,264,761.96	**PASS**
Average order value	1,053.97	1,053.97	**PASS**
Duplicate OrderIDs (rows returned)	0	0	**PASS**
Orders above IQR fence ($3,330.41)	8	8	**PASS**
Repeat customers	11	11	**PASS**
Cancelled + returned order value	519,673.91	519,673.91	**PASS**
Cancelled + returned orders	497	497	**PASS**
Median order value	823.62	823.62	**PASS**
Orders above average	491	491	**PASS**
H1 2023 revenue	286,501.52	286,501.52	**PASS**
H1 2024 revenue	257,059.34	257,059.34	**PASS**
H1 2025 revenue	231,882.85	231,882.85	**PASS**

Each SQL answer was recomputed with pandas from the same Excel file. The values also agree with the Project 2 report (total value, median, repeat customers, large orders, first-half revenue, cancelled/returned value).

A. Data overview

Q01. Preview the first 10 orders

Business question: *What does an order record look like?*

```
SELECT OrderID, Date, Product, Quantity, UnitPrice, TotalPrice, OrderStatus
FROM orders
ORDER BY Date, OrderID
LIMIT 10;
```

OrderID	Date	Product	Quantity	UnitPrice	TotalPrice	OrderStatus
ORD200112	2023-01-01	Monitor	1	$410.60	$410.60	Cancelled
ORD200236	2023-01-01	Monitor	1	$318.81	$318.81	Pending
ORD200848	2023-01-01	Chair	2	$645.85	$1,291.70	Shipped
ORD200373	2023-01-02	Laptop	3	$341.15	$1,023.45	Pending
ORD200645	2023-01-02	Laptop	2	$150.05	$300.10	Cancelled
ORD200881	2023-01-03	Tablet	2	$448.85	$897.70	Pending
ORD200000	2023-01-04	Monitor	5	$570.62	$2,853.10	Shipped
ORD200698	2023-01-04	Laptop	2	$388.22	$776.44	Delivered
ORD200371	2023-01-05	Phone	5	$307.76	$1,538.80	Pending
ORD200491	2023-01-05	Phone	5	$420.70	$2,103.50	Pending

Insight: Each order is one row: date, product, quantity, price, total and status. The first order in the data is dated 2023-01-01.

Q02. Headline business numbers

Business question: *How big is the business in this dataset?*

```
SELECT COUNT(*)                         AS total_orders,
       COUNT(DISTINCT CustomerID)       AS unique_customers,
       ROUND(SUM(TotalPrice), 2)        AS total_order_value,
       ROUND(AVG(TotalPrice), 2)        AS avg_order_value,
       MIN(TotalPrice)                  AS smallest_order,
       MAX(TotalPrice)                  AS largest_order,
       MIN(Date)                        AS first_order_date,
       MAX(Date)                        AS last_order_date
FROM orders;
```

Measure	Value
total_orders	1,200
unique_customers	1,189
total_order_value	$1,264,761.96
avg_order_value	$1,053.97
smallest_order	$11.39
largest_order	$3,456.40
first_order_date	2023-01-01
last_order_date	2025-06-30

Insight: 1,200 orders from 1,189 customers worth $1,264,761.96. The average order is $1,053.97, ranging from $11.39 to $3,456.40.

Q03. Integrity audit: duplicate OrderIDs

Business question: *Is every OrderID truly unique? (Project 1 gate, re-proved in SQL) Expected result: 0 rows.*

```
SELECT OrderID, COUNT(*) AS occurrences
FROM orders
GROUP BY OrderID
HAVING COUNT(*) > 1;
```

Result
0 rows returned (no duplicate OrderIDs exist)

Insight: Zero rows: **every OrderID is unique**. The table also enforces this itself (PRIMARY KEY on OrderID, UNIQUE on TrackingNumber), so a duplicate cannot be inserted.

B. Filtering with WHERE

Q04. Unusually large orders (above the IQR fence)

Business question: *Which orders are the extreme high-value ones? ($3,330.41 is the Project 2 IQR upper fence)*

```
SELECT OrderID, Date, Product, Quantity, UnitPrice, TotalPrice, OrderStatus
FROM orders
WHERE TotalPrice > 3330.41
ORDER BY TotalPrice DESC;
```

OrderID	Date	Product	Quantity	UnitPrice	TotalPrice	OrderStatus
ORD200789	2023-08-17	Tablet	5	$691.28	$3,456.40	Delivered
ORD201122	2023-06-07	Monitor	5	$678.19	$3,390.95	Returned
ORD200632	2023-05-02	Laptop	5	$678.16	$3,390.80	Delivered
ORD200469	2023-11-26	Chair	5	$676.98	$3,384.90	Cancelled
ORD200328	2023-02-28	Tablet	5	$674.04	$3,370.20	Cancelled
ORD200107	2023-03-27	Printer	5	$670.75	$3,353.75	Shipped
ORD200326	2024-07-01	Laptop	5	$670.48	$3,352.40	Returned
ORD201065	2023-10-30	Printer	5	$666.80	$3,334.00	Delivered

Insight: 8 orders exceed the fence, all with Quantity = 5. 7 of the 8 were placed in 2023 and 0 in 2025 (p = 0.013 if big orders were spread like all orders). Status: 3 delivered, 2 returned, 2 cancelled, 1 shipped. Worth monitoring; found after looking, so treat as a lead, not a conclusion.

Q05. Delivered 2024 orders that used a coupon

Business question: *What are the biggest successfully delivered coupon orders of 2024?*

```
SELECT OrderID, Date, Product, CouponCode, TotalPrice
FROM orders
WHERE OrderStatus = 'Delivered'
  AND Date BETWEEN '2024-01-01' AND '2024-12-31'
  AND CouponCode <> 'No Coupon'
ORDER BY TotalPrice DESC
LIMIT 10;
```

OrderID	Date	Product	CouponCode	TotalPrice
ORD200361	2024-06-29	Printer	SAVE10	$3,299.25
ORD200511	2024-03-15	Monitor	WINTER15	$2,876.20
ORD200578	2024-10-10	Monitor	SAVE10	$2,830.35
ORD200781	2024-01-10	Phone	FREESHIP	$2,621.30
ORD200558	2024-07-16	Phone	WINTER15	$2,512.48
ORD200994	2024-06-27	Tablet	WINTER15	$2,462.60
ORD200174	2024-04-16	Printer	SAVE10	$2,403.24
ORD200726	2024-10-25	Laptop	FREESHIP	$2,332.08
ORD201006	2024-05-27	Tablet	SAVE10	$2,104.90
ORD200178	2024-04-11	Chair	SAVE10	$2,048.24

Insight: The largest delivered coupon order of 2024 is $3,299.25 (Printer, SAVE10). 5 of these top 10 used SAVE10.

Q06. Pattern matching with LIKE and IN

Business question: *How do products starting with "P" sell when paid by card?*

```
SELECT Product, PaymentMethod,
       COUNT(*)                 AS orders,
       ROUND(SUM(TotalPrice), 2) AS order_value
FROM orders
WHERE Product LIKE 'P%'
  AND PaymentMethod IN ('Credit Card', 'Debit Card')
GROUP BY Product, PaymentMethod
ORDER BY order_value DESC;
```

Product	PaymentMethod	orders	order_value
Printer	Credit Card	39	$43,289.52
Printer	Debit Card	33	$33,165.09
Phone	Credit Card	35	$32,317.43
Phone	Debit Card	30	$31,731.75

Insight: Printer paid by Credit Card leads with 39 orders ($43,289.52). All four groups sit between 30 and 39 orders.

C. GROUP BY and aggregations

Q07. Orders, value and average by product

Business question: *Which products bring in the most order value?*

```
SELECT Product,
       COUNT(*)                                                    AS orders,
       ROUND(SUM(TotalPrice), 2)                                   AS order_value,
       ROUND(AVG(TotalPrice), 2)                                   AS avg_order_value,
       ROUND(100.0 * SUM(TotalPrice) / (SELECT SUM(TotalPrice) FROM orders), 1) AS pct_of_value
FROM orders
GROUP BY Product
ORDER BY order_value DESC;
```

Product	orders	order_value	avg_order_value	pct_of_value
Chair	178	$195,620.11	$1,098.99	15.5%
Printer	181	$195,612.61	$1,080.73	15.5%
Laptop	173	$192,126.56	$1,110.56	15.2%
Tablet	179	$186,568.95	$1,042.28	14.8%
Monitor	163	$175,651.41	$1,077.62	13.9%
Desk	170	$167,459.93	$985.06	13.2%
Phone	156	$151,722.39	$972.58	12.0%

Insight: Chair and Printer lead at 15.5% of value each; Phone is lowest at 12.0%. The gaps follow order counts (156 to 181) and are within chance (Project 2: p = 0.82).

Q08. Order status mix (percentage contribution)

Business question: *How much order value sits in each status?*

```
SELECT OrderStatus,
       COUNT(*)                                                    AS orders,
       ROUND(SUM(TotalPrice), 2)                                   AS order_value,
       ROUND(100.0 * SUM(TotalPrice) / (SELECT SUM(TotalPrice) FROM orders), 1) AS pct_of_value
FROM orders
GROUP BY OrderStatus
ORDER BY order_value DESC;
```

OrderStatus	orders	order_value	pct_of_value
Cancelled	250	$276,396.21	21.9%
Pending	237	$256,328.15	20.3%
Shipped	235	$246,159.58	19.5%
Returned	247	$243,277.70	19.2%
Delivered	231	$242,600.32	19.2%

Insight: Cancelled is the largest status at 21.9% of value ($276,396.21); Delivered is the smallest at 19.2%. The five statuses are almost evenly split, which is unusual for real data (see Limitations).

Q09. Revenue at risk (CASE grouping)

Business question: *What share of order value is delivered, still in flight, or lost to cancellations and returns?*

```
SELECT CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN 'At risk (Cancelled or Returned)'
            WHEN OrderStatus = 'Delivered'                THEN 'Delivered'
            ELSE 'In flight (Pending or Shipped)' END                    AS status_group,
       COUNT(*)                                                          AS orders,
       ROUND(SUM(TotalPrice), 2)                                         AS order_value,
       ROUND(100.0 * SUM(TotalPrice) / (SELECT SUM(TotalPrice) FROM orders), 1) AS pct_of_value
FROM orders
GROUP BY status_group
ORDER BY order_value DESC;
```

status_group	orders	order_value	pct_of_value
At risk (Cancelled or Returned)	497	$519,673.91	41.1%
In flight (Pending or Shipped)	472	$502,487.73	39.7%
Delivered	231	$242,600.32	19.2%

Insight: 41.1% of order value ($519,673.91 across 497 orders) is cancelled or returned. Only 19.2% ($242,600.32) is delivered; 39.7% is still in flight.

Q10. Payment method performance

Business question: *Does the way customers pay relate to order value?*

```
SELECT PaymentMethod,
       COUNT(*)                  AS orders,
       ROUND(SUM(TotalPrice), 2) AS order_value,
       ROUND(AVG(TotalPrice), 2) AS avg_order_value
FROM orders
GROUP BY PaymentMethod
ORDER BY order_value DESC;
```

PaymentMethod	orders	order_value	avg_order_value
Credit Card	234	$263,847.63	$1,127.55
Online	258	$262,442.94	$1,017.22
Cash	246	$259,786.29	$1,056.04
Gift Card	230	$246,323.92	$1,070.97
Debit Card	232	$232,361.18	$1,001.56

Insight: Credit Card has the highest total ($263,847.63) and average order ($1,127.55); Debit Card the lowest average ($1,001.56). Averages differ by under 13%, and Project 2 found no significant difference (p = 0.45).

Q11. Referral source performance

Business question: *Which marketing channels bring orders and value?*

```
SELECT ReferralSource,
       COUNT(*)                  AS orders,
       ROUND(SUM(TotalPrice), 2) AS order_value,
       ROUND(AVG(TotalPrice), 2) AS avg_order_value
FROM orders
GROUP BY ReferralSource
ORDER BY order_value DESC;
```

ReferralSource	orders	order_value	avg_order_value
Instagram	259	$275,285.45	$1,062.88
Email	250	$261,808.55	$1,047.23
Google	241	$250,441.48	$1,039.18
Facebook	228	$250,410.90	$1,098.29
Referral	222	$226,815.58	$1,021.69

Insight: Instagram brings the most orders (259) and value ($275,285.45); Referral the least. Facebook has the highest average order ($1,098.29). Not significant (Project 2: p = 0.89).

Q12. Coupon use versus no coupon

Business question: *Do orders with a coupon have a different average value?*

```
SELECT CASE WHEN CouponCode = 'No Coupon' THEN 'No coupon' ELSE 'Coupon used' END AS coupon_use,
       COUNT(*)                  AS orders,
       ROUND(SUM(TotalPrice), 2) AS order_value,
       ROUND(AVG(TotalPrice), 2) AS avg_order_value
FROM orders
GROUP BY coupon_use
ORDER BY orders DESC;
```

coupon_use	orders	order_value	avg_order_value
Coupon used	891	$942,360.55	$1,057.64
No coupon	309	$322,401.41	$1,043.37

Insight: 74.2% of orders (891) used a coupon, yet their average order ($1,057.64) is only $14.27 above no-coupon orders ($1,043.37). That gap is well within noise (Project 2 median test: p = 0.79).

D. Time trends

Q13. Yearly orders and value

Business question: *How does each calendar year compare? (2025 covers January to June only)*

```
SELECT strftime('%Y', Date)      AS year,
       COUNT(*)                  AS orders,
       ROUND(SUM(TotalPrice), 2) AS order_value,
       ROUND(AVG(TotalPrice), 2) AS avg_order_value
FROM orders
GROUP BY year
ORDER BY year;
```

year	orders	order_value	avg_order_value
2023	510	$552,643.24	$1,083.61
2024	459	$480,235.87	$1,046.27
2025	231	$231,882.85	$1,003.82

Insight: Orders fell from 510 (2023) to 459 (2024), -10.0%. 2025 has only six months (231 orders), so compare it with the first half only (next query).

Q14. First-half revenue, year over year (CTE + window function)

Business question: *Is like-for-like (Jan to Jun) revenue growing or shrinking?*

```
WITH h1 AS (
    SELECT strftime('%Y', Date) AS year,
           COUNT(*)             AS orders,
           SUM(TotalPrice)      AS revenue
    FROM orders
    WHERE CAST(strftime('%m', Date) AS INTEGER) <= 6
    GROUP BY strftime('%Y', Date)
)
SELECT year,
       orders,
       ROUND(revenue, 2) AS revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY year)) / LAG(revenue) OVER (ORDER BY year), 1) AS
           yoy_pct
FROM h1
ORDER BY year;
```

year	orders	revenue	yoy_pct
2023	252	$286,501.52	-
2024	237	$257,059.34	-10.3%
2025	231	$231,882.85	-9.8%

Insight: First-half revenue fell 10.3% in 2024 and 9.8% in 2025, identical to the Project 2 results. This is the fair like-for-like comparison.

Q15. Monthly orders and value

Business question: *What does the month-by-month pattern look like?*

```
SELECT strftime('%Y-%m', Date)   AS month,
       COUNT(*)                  AS orders,
       ROUND(SUM(TotalPrice), 2) AS order_value
FROM orders
GROUP BY month
ORDER BY month;
```

month	orders	order_value
2023-01	47	$56,685.75
2023-02	37	$40,117.66
2023-03	43	$48,609.37
2023-04	31	$27,751.71
2023-05	49	$63,836.84

month	orders	order_value
2023-06	45	$49,500.19
2023-07	44	$42,820.66
2023-08	51	$54,352.14
2023-09	29	$29,526.67
2023-10	47	$52,607.85
2023-11	41	$43,079.67
2023-12	46	$43,754.73
2024-01	32	$38,528.08
2024-02	32	$36,909.57
2024-03	36	$36,030.90
2024-04	50	$49,613.14
2024-05	34	$27,909.11
2024-06	53	$68,068.54
2024-07	43	$42,963.98
2024-08	28	$31,991.07
2024-09	44	$39,794.98
2024-10	31	$37,226.97
2024-11	35	$32,413.76
2024-12	41	$38,785.77
2025-01	27	$29,099.40
2025-02	37	$35,317.55
2025-03	49	$39,200.66
2025-04	32	$31,821.20
2025-05	37	$43,396.64
2025-06	49	$53,047.40

Insight: The best month is 2024-06 ($68,068.54) and the weakest is 2023-04 ($27,751.71). Monthly revenue swings widely, which is why a straight-line trend was not statistically significant in Project 2 ($p = 0.11$).

E. HAVING and subqueries

Q16. Products with an above-average cancel/return rate

Business question: *Which products lose more orders than the business overall?*

```
SELECT Product,
       COUNT(*) AS orders,
       SUM(CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN 1 ELSE 0 END) AS at_risk_orders,
       ROUND(100.0 * SUM(CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN 1 ELSE 0 END) / COUNT(*), 1) AS
            at_risk_pct
FROM orders
GROUP BY Product
HAVING 100.0 * SUM(CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN 1 ELSE 0 END) / COUNT(*)
       > (SELECT 100.0 * SUM(CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN 1 ELSE 0 END) / COUNT(*)
           FROM orders)
ORDER BY at_risk_pct DESC;
```

Product	orders	at_risk_orders	at_risk_pct
Monitor	163	71	43.6%
Tablet	179	77	43.0%
Laptop	173	74	42.8%

Insight: Monitor (43.6%), Tablet (43.0%), Laptop (42.8%) sit above the overall 41.4% rate, but only by 1 to 2 points. **A chi-square test across all seven products gives p = 0.98, so there is no evidence that any product loses more orders than another.** HAVING shows what a result set looks like; statistics say whether to act on it.

Q17. Repeat customers

Business question: *Which customers ordered more than once?*

```
SELECT CustomerID,
       COUNT(*)                 AS orders,
       ROUND(SUM(TotalPrice), 2) AS total_value,
       MIN(Date)                AS first_order,
       MAX(Date)                AS last_order
FROM orders
GROUP BY CustomerID
HAVING COUNT(*) > 1
ORDER BY total_value DESC;
```

CustomerID	orders	total_value	first_order	last_order
C38840	2	$5,723.23	2023-06-07	2024-06-08
C97593	2	$2,855.22	2023-05-03	2024-02-22
C98474	2	$2,175.34	2023-03-27	2024-06-03
C70659	2	$1,853.96	2023-08-20	2025-03-01
C94569	2	$1,675.35	2024-03-23	2024-06-21
C46651	2	$1,360.89	2023-08-01	2024-03-20
C35852	2	$1,248.39	2024-03-05	2025-03-02
C14847	2	$1,097.81	2023-01-01	2024-12-19
C91155	2	$704.82	2023-07-17	2024-09-08
C21191	2	$647.77	2024-04-11	2025-05-16
C56969	2	$193.00	2023-06-29	2024-01-09

Insight: Only 11 customers ordered more than once (0.9% of 1,189). The largest, C38840, spent $5,723.23 over two orders. Either repeat buying is rare or CustomerID is not a stable customer key.

Q18. Orders above the average order value

Business question: *How many orders are larger than the "average" order?*

```
SELECT COUNT(*)                                                    AS orders_above_average,
       ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM orders), 1)  AS pct_of_all_orders,
       ROUND((SELECT AVG(TotalPrice) FROM orders), 2)              AS average_order_value
FROM orders
WHERE TotalPrice > (SELECT AVG(TotalPrice) FROM orders);
```

orders_above_average	pct_of_all_orders	average_order_value
491	40.9%	$1,053.97

Insight: 491 orders (40.9%) exceed the $1,053.97 average, while by definition 50% exceed the median. A long tail of large orders pulls the average above the typical order.

Q19. Median order value in pure SQL

Business question: *What is the typical (median) order value? SQLite has no MEDIAN(), so the middle row(s) are picked.*

```
SELECT ROUND(AVG(TotalPrice), 2) AS median_order_value
FROM (
    SELECT TotalPrice
    FROM orders
    ORDER BY TotalPrice
    LIMIT 2 - (SELECT COUNT(*) FROM orders) % 2
    OFFSET (SELECT (COUNT(*) - 1) / 2 FROM orders)
);
```

median_order_value
$823.62

Insight: The pure-SQL median is **$823.62**, identical to the Python result in Project 2, and $230.35 below the average. Use the median to describe a typical order.

F. Business insight

Q20. Cancelled or returned orders by year

Business question: *Is the cancel/return problem getting better or worse?*

```
SELECT strftime('%Y', Date) AS year,
       COUNT(*)             AS orders,
       SUM(CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN 1 ELSE 0 END)                    AS
           at_risk_orders,
       ROUND(100.0 * SUM(CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN 1 ELSE 0 END) / COUNT(*), 1) AS
            at_risk_pct,
       ROUND(SUM(CASE WHEN OrderStatus IN ('Cancelled', 'Returned') THEN TotalPrice ELSE 0 END), 2) AS
           at_risk_value
FROM orders
GROUP BY year
ORDER BY year;
```

year	orders	at_risk_orders	at_risk_pct	at_risk_value
2023	510	209	41.0%	$222,095.51
2024	459	180	39.2%	$179,895.94
2025	231	108	46.8%	$117,682.46

Insight: The cancelled-or-returned rate was 41.0% (2023), 39.2% (2024) and 46.8% (first half of 2025). Project 2 found the change across years not conclusive (p = 0.11), so keep monitoring.

Q21. What changed in the average order: quantity, price or big orders?

Business question: *Did customers buy fewer units, pay lower prices, or place fewer large ($2,500+) orders over time?*

```
SELECT strftime('%Y', Date) AS year,
       COUNT(*)             AS orders,
       ROUND(AVG(Quantity), 2)   AS avg_quantity,
       ROUND(AVG(UnitPrice), 2)  AS avg_unit_price,
       ROUND(AVG(TotalPrice), 2) AS avg_order_value,
       SUM(CASE WHEN TotalPrice >= 2500 THEN 1 ELSE 0 END) AS large_orders,
       ROUND(100.0 * SUM(CASE WHEN TotalPrice >= 2500 THEN 1 ELSE 0 END) / COUNT(*), 1) AS large_order_pct
FROM orders
GROUP BY year
ORDER BY year;
```

year	orders	avg_quantity	avg_unit_price	avg_order_value	large_orders	large_order_pct
2023	510	2.99	$358.68	$1,083.61	43	8.4%
2024	459	2.94	$352.24	$1,046.27	37	8.1%
2025	231	2.86	$359.70	$1,003.82	14	6.1%

Insight: Average quantity (2.99 to 2.86) and unit price ($352.24 to $359.70) barely move, while the share of $2,500+ orders slips from 8.4% to 6.1%. **Order value does not differ significantly by year (Kruskal-Wallis p = 0.64)**, so treat the lower average order as a signal to watch, not a proven change.

Limitations

- **One table, so no JOIN.** The dataset has a single flat table. With separate customer and product tables, the same queries would JOIN them.
- **SQLite dialect.** strftime() and the median pattern are SQLite-specific; MySQL uses DATE_FORMAT() / YEAR(), PostgreSQL uses EXTRACT() / PERCENTILE_CONT().
- **Same caveats as Project 2.** Status, product, channel and weekday are unusually evenly spread, and CustomerID is almost never repeated; confirm both at the source before acting.
- **SQL describes; it does not prove.** Gaps between groups were tested for significance before being called real (Q16, Q21).

How to Reproduce

- Put **run_sql_analysis.py**, **queries.sql** and **Cleaned_Dataset.xlsx** in one folder.
- Install once: **py -m pip install pandas openpyxl**. Run: **py run_sql_analysis.py**. It rebuilds orders.db, SQL_Results.xlsx and sql_results.json.
- To explore by hand, open **orders.db** in DB Browser for SQLite (or the VS Code SQLite extension), paste any query from **queries.sql** and press run.

Deliverables

SQL_Analysis_Report_Project3.pdf (this report) | **queries.sql** (21 queries) | **orders.db** (SQLite database) | **SQL_Results.xlsx** (every result, plus the verification sheet) | **run_sql_analysis.py** (re-runnable pipeline) | **README.md**